EEL 5764: Computer Architecture

Lectures 13–14: Research Frontiers & Project Exploration

Sandip Ray

Warren B. Nelms Endowed Professor

Department of Electrical & Computer Engineering, University of Florida | Office: MALA 4131

Required Reading: H&P Chapter 5 & Appendix C, Selected Research Papers

The Evolving Landscape of Computer Architecture

THE TRANSITION FROM FREQUENCY SCALING TO DOMAIN SPECIFICITY

THE PHYSICAL & ARCHITECTURAL WALLS

Breakdown of Dennard Scaling: Voltage scaling hit sub-micron leakage limits (~1.0V floor), ending the era of "free" clock frequency growth.

The Power & Memory Walls: Thermal dissipation caps single-chip power budgets, while DRAM latency continues to lag behind CPU cycle speeds.

Dark Silicon Phenomenon: A significant percentage of physical silicon transistors must remain unpowered or dimmed to stay within TDP limits.

THE PARADIGM SHIFT TO ACCELERATION

Domain-Specific Architectures (DSAs): Tailoring hardware datapaths to specific workload classes (matrices, vectors, graphs) achieves 10x-100x efficiency gains.

Heterogeneous Integration: Modern SoCs combine general-purpose CPUs, massive SIMD GPUs, spatial neural processing units, and high-speed interconnects.

Hardware-Software Co-Design: Compilers, ISAs, and microarchitectures are co-optimized to exploit data-level and thread-level parallelism.

Research Frontier 1: GPUs & SIMT Parallelism

SIMT EXECUTION MODEL

Single Instruction Multiple Threads: Groups of threads (warps) execute the same instruction across parallel SIMD lanes.

Zero-Overhead Context Switching: Hardware register files preserve state for thousands of active threads to hide memory stalls.

Thread Block Scheduling: Streaming Multiprocessors (SMs) dynamically assign work blocks to maximize compute density.

HIGH-BANDWIDTH MEMORY

HBM3 Stacked DRAM: 3D stacked memory stacks connected via silicon interposers deliver multi-terabyte/sec memory bandwidth.

Coalesced Memory Accesses: Hardware memory controllers combine adjacent thread requests into unified 128-byte cache lines.

Memory Wall Pressure: AI and graph workloads demand massive memory bandwidth to keep thousands of execution units fed.

RESEARCH CHALLENGES

Control Divergence: Conditional branches (if-else) force warp serialization, causing severe lane idling penalties.

Uncoalesced Memory Stalls: Irregular pointer chasing causes memory bus fragmentation and memory controller thrashing.

Heterogeneous CPU-GPU Sync: Coherence protocols and unified virtual memory (UVM) page-table migrations.

Research Frontier 2: AI Accelerators & Systolic Arrays

SYSTOLIC ARRAY DATAFLOW

- **2D Processing Element (PE) Grids:** Data flows rhythmically across neighboring MAC units without returning to a central register file.
- **Weight-Stationary Dataflow:** Weights remain frozen in PE registers while activations stream horizontally, minimizing DRAM accesses.
- **Output-Stationary Dataflow:** Partial sums accumulate locally in PEs while inputs and weights stream across the array.

QUANTIZATION & SPARSITY

- **Low-Precision Arithmetic:** Transitioning from FP32 to INT8, FP16, and MXFP4 reduces silicon area and dynamic switching energy.
- **Structured Sparsity:** Hardware skipping of zero-valued weights/activations (e.g. 2:4 structured sparsity) doubles throughput.
- **Energy-Delay Product (EDP):** Matrix multiplication acceleration yields orders of magnitude higher EDP efficiency over general CPUs.

RESEARCH CHALLENGES

- **Array Size vs. Bandwidth:** Scaling PE grids (32x32 to 128x128) creates massive scratchpad SRAM bandwidth bottlenecks.
- **Non-Linear Activation Units:** Hardware execution of Softmax, LayerNorm, and GELU outside the matrix array.
- **Reconfigurability:** Balancing fixed-function ASIC efficiency with flexible support for evolving AI model topologies.

Research Frontier 3: The RISC-V Ecosystem & Extensibility

THE OPEN-STANDARD INSTRUCTION SET ARCHITECTURE REVOLUTION

MODULAR ISA ARCHITECTURE

Base Integer Set (RV32I/RV64I): A minimal, highly optimized 40-instruction base contract that every hardware core must implement.

Standard Extensions: Modular addition of Multiplication (M), Atomics (A), Floating-Point (F/D), Compressed (C), and Vector (V) features.

Royalty-Free Freedom: Eliminates proprietary IP licensing barriers, enabling academic researchers to modify the ISA directly.

CUSTOM EXTENSION RESEARCH

Reserved Custom Opcodes: Dedicated bit-space in the RISC-V opcode map reserved specifically for domain-specific instructions.

Custom Instruction Synthesis: Tightly coupling specialized hardware functional units (e.g., cryptographic, tensor) directly into the CPU pipeline.

Toolchain Co-Design: Modifying GCC/LLVM compilers and gem5 simulators to recognize custom instructions and generate optimal binaries.

Research Frontier 4: Chiplets & 2.5D/3D Packaging

SILICON YIELD ECONOMICS

Reticle Size Limits: Large monolithic dies hit physical lithography bounds (~850 mm²) and suffer exponential defect yield loss.

Modular Disaggregation: Breaking a monolithic chip into smaller, specialized chiplet dies dramatically improves wafer yield.

Heterogeneous Process Nodes: Manufacturing compute cores on 3nm while keeping I/O and analog controllers on cost-effective 12nm nodes.

INTERPOSER INTERCONNECTS

2.5D Silicon Interposers: High-density passive or active silicon substrates enabling fine-pitch microbump connections between dies.

3D Vertical Stacking: Direct die-to-die stacking using Through-Silicon Vias (TSVs) for ultra-low latency cache stacks (e.g. 3D V-Cache).

UCIe Open Standard: Universal Chiplet Interconnect Express establishing standardized physical layer protocols for multi-die communication.

RESEARCH CHALLENGES

Die-to-Die Latency Penalty: Inter-chiplet communication adds physical wire propagation delays over monolithic single-die routing.

Cross-Chiplet NoC Topologies: Designing routing protocols and credit-based flow control for multi-die Networks-on-Chip.

Thermal Density & Delivery: Dissipating heat from stacked 3D dies and managing power delivery network (PDN) IR drop.

Research Frontier 5: Hardware Security & Microarchitectural Trust

SIDE-CHANNEL ATTACKS

Cache Timing Attacks: Measuring cache hit vs. miss latency (Prime+Probe, Flush+Reload) to infer secret cryptographic keys.

Power & Electromagnetic Analysis: Monitoring physical supply current variations to extract internal register state.

Shared Resource Contention: Exploit paths in shared SMT threads, branch predictors, and memory controllers.

TRANSIENT EXECUTION LEAKS

Speculative Execution Exploits: Attacks like Spectre and Meltdown trick branch predictors into speculatively executing illegal reads.

Microarchitectural State Footprints: Speculative loads modify L1 cache state before exception checks roll back architectural registers.

Architectural vs. Microarchitectural State: Security guarantees must extend beyond the ISA contract down to internal hardware buffers.

RESEARCH MITIGATIONS

Secure Cache Partitioning: Way-partitioning and randomized mapping caches to prevent timing leakage between security domains.

Hardware Enclaves: Secure execution environments (e.g. RISC-V Keystone, ARM TrustZone) with hardware memory encryption.

Post-Silicon Validation: Verifying that microarchitectural optimizations do not create unintended side-channel leakage paths.

Research Frontier 6: Heterogeneous SoCs & Shared Memory

UNIFIED MEMORY ARCHITECTURES & HARDWARE COHERENCE

UNIFIED MEMORY ARCHITECTURES (UMA)

Eliminating Copy Overheads: CPU and accelerator cores share a high-bandwidth, unified physical memory pool (e.g., Apple M-series, AMD APUs).

Zero-Copy Pointer Passing: High-level software passes virtual memory pointers directly to hardware accelerators without PCIe host-to-device transfers.

Shared Page Tables: System MMUs enable hardware accelerators to participate directly in virtual memory translation and page fault handling.

HARDWARE COHERENCE & INTERCONNECTS

Heterogeneous Cache Coherence: Extending directory-based or snooping coherence protocols across CPU L2/L3 caches and accelerator scratchpads.

Network-on-Chip (NoC) Topologies: Designing mesh, torus, or crossbar NoCs to handle concurrent high-bandwidth streams without deadlocks.

Memory-Level Parallelism (MLP): Balancing memory controller request queues between latency-sensitive CPU reads and throughput-bound accelerator bursts.

Introduction to the Semester Research Project

BRIDGING THEORETICAL ARCHITECTURE WITH EMPIRICAL RESEARCH

PROJECT PURPOSE & VISION

Core Objective: Conduct an "original", empirical microarchitectural research study using the gem5 simulation environment.

Substantive Research Experience: Move beyond textbook exercises to formulate a research hypothesis, implement hardware modifications, and evaluate performance.

Weight & Importance: Accounts for 40% of your overall course grade, serving as the central hands-on milestone of EEL 5764.

WHAT CONSTITUTES A STRONG PROJECT?

Focused Scope: A clearly defined architectural mechanism rather than a vague or overly broad survey.

Empirical Evaluation: Quantitative trade-off analysis measuring CPI, execution time, cache miss rates, or energy-delay product.

Systematic Comparison: Comparing a modified experimental design against a well-defined, realistic baseline.

Example Project Track 1: Core & Pipeline Microarchitecture

MICROARCHITECTURAL INNOVATIONS IN PROCESSOR EXECUTION PIPELINES

POTENTIAL RESEARCH TOPICS

Branch Predictor Comparative Analysis: Evaluating static, 2-bit local, Tournament, and TAGE branch predictors across SPEC benchmarks in gem5.

Instruction Window Sizing: Analyzing how Reorder Buffer (ROB) and issue queue capacity impact IPC under varying memory stall latencies.

Forwarding Path Trade-Offs: Measuring the quantitative IPC loss when selectively disabling specific hardware forwarding paths in deep pipelines.

GEM5 IMPLEMENTATION & METRICS

Target CPU Models: Configure parameters in `TimingSimpleCPU` or `O3CPU` using `--bp-type` and pipeline capacity flags.

Key Telemetry Metrics: Parse `m5out/stats.txt` for `system.cpu.cpi`, `condIncorrect`, `statCyclesStalledByMem`, and `ROBFullStalls`.

Expected Outcome: Characterize the performance vs. hardware complexity trade-offs of pipeline depth and dynamic instruction scheduling.

Example Project Track 2: Memory Hierarchy & Cache Innovation

ADDRESSING THE MEMORY WALL THROUGH CACHE & PREFETCH DESIGN

POTENTIAL RESEARCH TOPICS

Data Prefetching Algorithms: Evaluating stride, stream, and tagged hardware prefetchers under memory-intensive application benchmarks.

Cache Replacement Policies: Comparing LRU, Random, and LFU replacement algorithms across direct-mapped vs. set-associative L2 caches.

Miss Status Holding Registers (MSHRs): Analyzing how MSHR depth affects memory-level parallelism (MLP) and cache stall cycles.

GEM5 IMPLEMENTATION & METRICS

Target Subsystems: Configure `Classic Cache` or `Ruby Memory` hierarchies using `--l1d_size`, `--l2cache`, and `--prefetcher` flags.

Key Telemetry Metrics: Parse `m5out/stats.txt` for `overall_miss_rate`, `overall_misses`, `avg_miss_latency`, and `AMAT`.

Expected Outcome: Demonstrate how memory hierarchy tuning reduces Average Memory Access Time (AMAT) and processor idle stalls.

Example Project Track 3: Domain Acceleration & Systolic Arrays

SPECIALIZED COMPUTING UNITS FOR AI, TENSOR, & VECTOR WORKLOADS

POTENTIAL RESEARCH TOPICS

Systolic Array Dataflow Exploration: Comparing Weight-Stationary, Output-Stationary, and Input-Stationary dataflows in 2D GEMM accelerators.

Array Scaling vs. Memory Bandwidth: Scaling NxN PE grids (8x8 to 32x32) to quantify the exact point where scratchpad SRAM bandwidth saturates.

RISC-V Vector (RVV) vs. Spatial Hardware: Evaluating vector-length agnostic (VLA) execution against dedicated 2D spatial array hardware.

GEM5 IMPLEMENTATION & METRICS

Target Subsystems: RISC-V Vector ISA models, custom SystemC/C++ accelerator SimObjects, or gem5-NVDLA/SLA interfaces.

Key Telemetry Metrics: Extract `sim_ticks`, `sim_insts`, speedup over scalar CPU baselines, and energy-delay product (EDP).

Expected Outcome: Quantify the 10x-100x efficiency gains of specialized 2D array dataflows over general-purpose instruction streams.

Example Project Track 4: Hardware Security & Side-Channels

EVALUATING MICROARCHITECTURAL TRUST & VULNERABILITY MITIGATIONS

POTENTIAL RESEARCH TOPICS

Speculative Execution Side-Channels: Modeling Spectre/Meltdown transient leakage paths in gem5 pipelines.

Secure Cache Partitioning: Evaluating way-partitioning or randomized mapping caches to prevent timing leaks.

Hardware Enclave Performance Tax: Measuring the execution latency overheads of memory encryption and secure state rollbacks.

GEM5 IMPLEMENTATION & METRICS

Target Subsystems: Configure `O3CPU` with speculative branch predictors and modified cache line access policies.

Key Telemetry Metrics: Track speculative instruction flushes, cache miss timing variations, and overhead percentage vs. standard baseline.

Expected Outcome: Demonstrate the performance-security trade-off curve in modern microarchitectural design.

Example Project Track 5: Ambitious / Stretch Goal Ideas [AMBITIOUS]

CO-DESIGN & COHERENCE [AMBITIOUS]

Out-of-Order CPU + Coherent Accelerator Co-Design: Building a custom C++/SystemC GEMM accelerator in gem5 with full hardware cache coherence.

Shared Memory Pointer Passing: Measuring data transfer overheads of zero-copy pointer passing vs. traditional DMA transfers.

High Complexity: Requires deep C++ modification of gem5 SimObjects and Ruby memory coherence controllers.

SPECULATION DEFENSE [AMBITIOUS]

Hardware Speculation Barrier Mechanism: Modifying the gem5 `O3CPU` pipeline to dynamically block speculative cache line loads on unresolved branches.

Zero-Leakage Guarantee: Proving zero Spectre-v2 transient leakage while constraining the performance penalty to <5%.

High Complexity: Requires modifying the Reorder Buffer (ROB) and Load-Store Queue (LSQ) execution logic.

CHIPLET & NOC TOPOLOGY [AMBITIOUS]

Multi-Die Chiplet NoC Interconnect Simulation: Modeling 2.5D interposer die-to-die latency using gem5's Garnet Network-on-Chip simulator.

Interposer Bandwidth Sweeps: Evaluating interposer bandwidth saturation across 4-chiplet topology sweeps (mesh vs. torus).

High Complexity: Requires detailed Ruby/Garnet configuration and multi-chiplet memory mapping setup.

Project Methodology & Experimental Rigor in gem5

1. BASELINE COMPARISON

Well-Defined Baselines: Establish an unmodified, standard gem5 CPU/cache configuration before applying hardware changes.

Controlled Parameters: Keep all secondary system parameters (clock rate, RAM latency, binary code) strictly constant.

Isolation of Variables: Ensure measured performance deltas stem solely from the architectural mechanism under test.

2. PARAMETER SWEEPS

Multi-Point Sweeps: Evaluate mechanisms across a range of parameters (e.g. cache size 16KB to 128KB, ROB 32 to 192 entries).

Identifying Knee of the Curve: Pinpoint diminishing returns and saturation points in microarchitectural trade-offs.

Benchmark Diversity: Test mechanisms across both compute-bound and memory-bound benchmark workloads.

3. QUANTITATIVE PLOTTING

Rigorous Telemetry Parsing: Extract exact statistics from `m5out/stats.txt` using Python automation scripts.

Publication-Quality Visuals: Plot speedup, CPI, miss rates, and energy-delay products with clearly labeled axes and units.

Analytical Discussion: Explain *why* the hardware behaved as observed using underlying microarchitectural principles.

Choosing a Strong Research Topic: The 4 Core Questions

1. THE RESEARCH QUESTION

What specific architectural bottleneck or inefficiency are you addressing?

Why does current standard hardware fail or incur overhead on this workload?

2. THE PROPOSED MECHANISM

What is the specific hardware structure, algorithm, or policy you are introducing?

How does it physically alter the pipeline, cache, or interconnect flow?

3. THE HARDWARE COST

What is the silicon area, wire complexity, or latency tax of your proposal?

Does your mechanism add gate delay to the critical path?

4. THE GEM5 METHODOLOGY

Which gem5 CPU model, cache configuration, or flags will you use?

What exact metric strings in `m5out/stats.txt` will prove your hypothesis?